Project Specifications

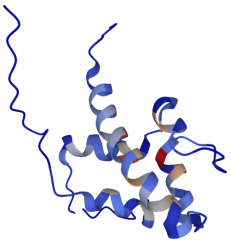
Structural Bioinformatics - University of Padova

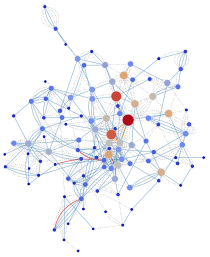
Classification of contacts in protein structures

Introduction

Residue Interaction Networks are derived from protein structures based on geometrical and physico-chemical properties of the amino acids. RING (https://ring.biocomputingup.it/) is a software that takes a PDB (https://www.rcsb.org/) file as input and returns the list of **contacts** (residue-residue pairs) and their **types** in a protein structure. RING contact types include:

- Hydrogen bonds (HBOND)
- Van der Waals interactions (VDW)
- Disulfide bridges (SBOND)
- Salt bridges (IONIC)
- π-π stacking (PIPISTACK)
- π-cation (PICATION)
- Hydrogen-alogen (HALOGEN)
- Metal ion coordination (METAL_ION)
- π-hydrogen bond (PIHBOND)
- *Unclassified contacts*





Each group of students has been tasked with developing **software** that can predict the RING classification of a contact based on statistical or supervised methods, rather than geometrical constraints. The program should be able to calculate the propensity (or probability) of a contact belonging to each of the different contact types defined by RING, starting from the protein structure.

In addition to the software, each group of students is requested to provide a **report** that describes their implementation and the results obtained from analyzing the provided dataset. The report should also include interesting examples that demonstrate the correctness of the calculation and help to convince the reader of the accuracy of the results.

Data and code

- classification_ring.zip - Feature extractions software along with a naive predictor as an example
- features_ring.zip - Training data archive

Features extraction

I provide two Python scripts *calc_features.py* and *calc_3di.py* that can be used to extract some features from PDB files and 3D contacts with the built-in BioPython modules.

The first script calculates structural properties (secondary structure, relative solvent accessibility, half-sphere exposure, Atchely's scales). The second script converts the PDB sequence into the 3Di alphabet used by FoldSeek.

The scripts can be executed with the following command:

```
$python3 calc_features.py my_pdb.cif -out_dir output_features/
$python3 calc_3di.py my_pdb.cif -out_dir output_3di/
```

Note that *calc_features.py* requires the **DSSP** program that only works on POSIX operating systems (not on Windows). Please check the paths in the *configuration.json* file.
Also note that *calc_3di.py* contains a variable that defines the position of the model files you need to modify depending on where you execute it.

All or some of the features calculated by the program can be used for training. Additional features can be calculated, if desired. Some features, such as the secondary structure, capture the local environment of the residues, while others, like Atchley, capture the physico-chemical properties of the amino acids. The 3Di FoldSeek encoding is calculated looking at the local environment of the contact in the 3D space.

Training set

Training data is provided in separate files, one for each PDB structure that has been generated by executing the two feature scripts (see above) and merging their outputs. All the files together (**3,914 PDBs**) contain the following numbers of **contacts**:

Contact type	Count
HBOND	1,055,929
VDW	737,061
PIPISTACK	38,283
IONIC	35,391
PICATION	8,885
SSBOND	2,100
PIHBOND	1,790
Unclassified	1,089,547

Every file is a **tab-separated table** with the columns below. Columns specify residue identifiers (same fields used in BioPython), pre-calculated features and the type of interaction (last column). This table differs from the one generated by the provided *calc_features.py* program just by the “Interaction” column (last column).

Column name	Column meaning	Type of column
pdb_id		
s_ch	chain	source residue identifier
s_resi	index	
s_ins	insertion code	
s_resn	name	
s_ss8	secondary structure 8 states (DSSP)	source residue features
s_rsa	relative solvent accessibility	
s_phi	phi angle	
s_psi	psi angle	
s_a1	Atchley feature 1	
s_a2	Atchley feature 2	
s_a3	Atchley feature 3	
s_a4	Atchley feature 4	
s_a5	Atchley feature 5	
s_3di_state	3Di state	
s_3di_letter	3Di alphabet	
t_ch	chain	target residue identifier
t_resi	index	
t_ins	insertion code	
t_resn	name	
t_ss8	secondary structure 8 states (DSSP)	target residue features
t_rsa	relative solvent accessibility	
t_phi	phi angle	
t_psi	psi angle	
t_a1	Atchley feature 1	
t_a2	Atchley feature 2	
t_a3	Atchley feature 3	
t_a4	Atchley feature 4	
t_a5	Atchley feature 5	
t_3di_state	3Di state	
t_3di_letter	3Di alphabet	
Interaction	interaction type	

Predictor

The predictor software requires a **PDB file** as input, extract the features and generates an output **table** including the **contact identifiers**, their **features**, the **interaction type** and the **score** (one line per contact; tab separated columns):

```
Source_residue_clumns (chain, index, insertion code, name)
Target_residue_columns (chain, index, insertion code, name)
Features_columns (...)
Interaction (hbond/vdw/...)
Score
```

Training can be performed by generating different **individual models** for each type of contact, i.e. training each class separately or as a **multi-class model**. Negative examples can be contacts of a different type (for single-class models). Alternative strategies can also be adopted. An example of a

Naive Bayes classifier called *classifier.ipynb* is provided in the same archive as the *calc_features.py* program.

Project requirements

1. A **software** implemented in Python3 for the prediction of contact types given a protein structure. The use of the BioPython.PDB, Argparse and Logging modules is encouraged. All paths to data files (e.g. models), paths to external tools (e.g. DSSP) and hyperparameters have to be provided in a configuration file. Use of the RING software is forbidden and will be checked by the teacher.
2. The trained **models** and the **script** (or program) to generate (train) them. Training has to be fully reproducible. Model evaluation can be provided using data provided in the training set and performing cross-validation.
3. A **report** that provides a detailed explanation of the adopted strategy (algorithm, etc.), statistics of the features (distribution, correlation, etc.), and an evaluation of the software. The report should be provided in PDF format and be between two and five pages of text, excluding figures and supporting documentation.
4. **Documentation** of the software, including information about how to use it and instructions about how to re-train the models. The documentation should be provided in **Markdown** language.

Project evaluation

- The algorithm will be blindly tested against a validation dataset not available to the students. Matthew's correlation coefficient, balanced accuracy, average precision score and area under the ROC will be considered. The predictor **performance** should be significantly better than random.
- **Reproducibility** of the training will be evaluated along with the performance.
- Clarity of the **documentation**, reproducibility of the results, and the quality of the reporting will also be evaluated.